

Connect with me → <https://www.linkedin.com/in/sakshamarora9575/>
<https://topmate.io/sakshamarora>

Problem Statement: AI-Powered Restaurant Recommendation System (Zomato Use Case)

You are tasked with building an **AI-powered restaurant recommendation service** inspired by Zomato. The system should intelligently suggest restaurants based on user preferences by combining structured data with a Large Language Model (LLM).

Objective

Design and implement an application that:

- Takes user preferences (such as location, budget, cuisine, and ratings)
- Uses a real-world dataset of restaurants
- Leverages an LLM to generate personalized, human-like recommendations
- Displays clear and useful results to the user

System Workflow

1. Data Ingestion

- Load and preprocess the Zomato dataset from Hugging Face (<https://huggingface.co/datasets/ManikaSaini/zomato-restaurant-recommendation>)
- Extract relevant fields such as restaurant name, location, cuisine, cost, rating, etc.

2. User Input

- Collect user preferences:
 - Location (e.g., Delhi, Bangalore)
 - Budget (low, medium, high)
 - Cuisine (e.g., Italian, Chinese)
 - Minimum rating
 - Any additional preferences (e.g., family-friendly, quick service)

3. Integration Layer

- Filter and prepare relevant restaurant data based on user input
- Pass structured results into an LLM prompt
- Design a prompt that helps the LLM reason and rank options

4. Recommendation Engine

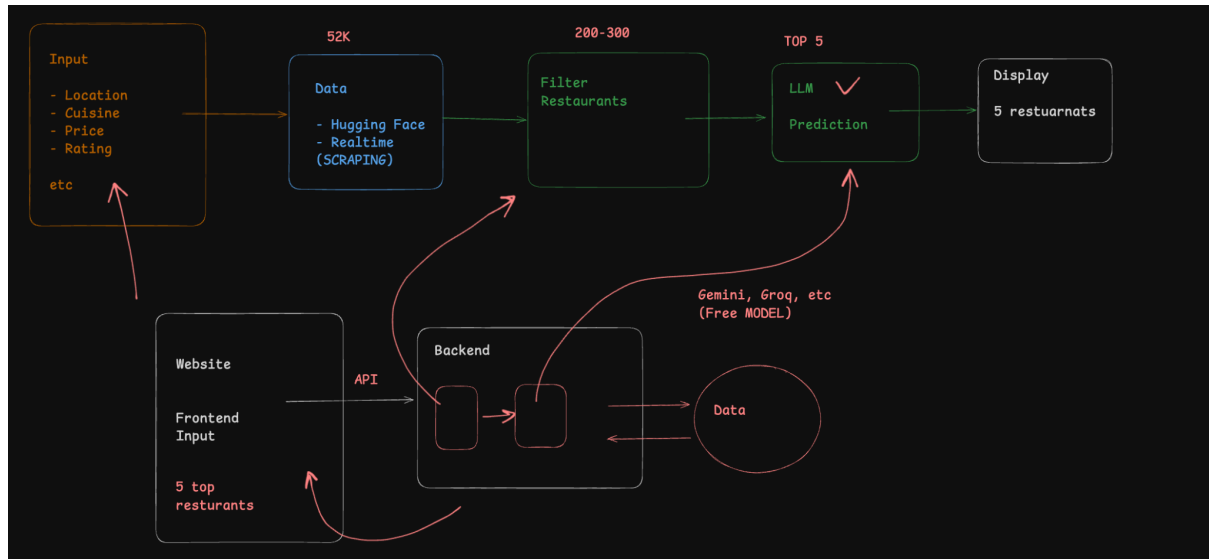
- Use the LLM to:
 - Rank restaurants
 - Provide explanations (why each recommendation fits)
 - Optionally summarize choices

5. Output Display

- Present top recommendations in a user-friendly format:
 - Restaurant Name

- Cuisine
- Rating
- Estimated Cost
- AI-generated explanation

Diagram →



Architecture

Phase-wise architecture: restaurant recommendation system

This document breaks the build into **phases** that map to the workflow in [problemstatement.md](#): data ingestion → user input → integration (filter + prompt prep) → LLM recommendation → output display.

Phase 0 — Scope and foundations

Item	Outcome
Product slice	Basic web UI — source of user input and primary presentation of results for milestone 1 (see phase0-scope.md); CLI remains for dev/diagnostics.
Stack	Language/runtime, dependency manager, where secrets live (e.g. .env for API keys, never committed).
Dataset contract	Confirm Hugging Face dataset fields you will support in v1; document column → internal field mapping.

Non-goals	Explicitly defer (e.g. user accounts, live Zomato API, maps) to avoid scope creep.
-----------	--

Exit criteria: written assumptions (stack, v1 UI, supported preference fields) and a local way to run the app end-to-end once later phases exist.

Implemented artifacts: package `src/milestone1/phase0/` (paths, scope, info/doctor commands), [phase0-scope.md](#), [dataset-contract.md](#), repo [README.md](#), `.env.example`. CLI: `milestone1 info / milestone1 doctor`.

Phase 1 — Data ingestion and canonical model

Layer	Responsibility
Acquisition	Download or stream ManikaSaini/zomato-restaurant-recommendation; cache locally if useful for iteration.
Normalization	Clean types (ratings as numbers, cost as enum or numeric band), handle missing values, dedupe rows if needed.
Canonical schema	Internal Restaurant (or equivalent) with: name, location, cuisines, cost, rating, plus any extra columns you keep for prompts.

Exit criteria: a single module (or package) that loads data and returns a typed in-memory collection or queryable table; unit tests on parsing for a few sample rows.

Implemented: package `src/milestone1/phase1_ingestion/` (Restaurant, load_restaurants / iter_restaurants, normalization, Hub revision pin, schema assertion). CLI: `milestone1 ingest-smoke --limit N`. Hub integration tests: `RUN_HF_INTEGRATION=1 pytest -m integration`.

Phase 2 — User preferences and validation

Component	Responsibility
Preference model	Structured fields: location, budget band, cuisine(s), minimum rating; optional free-text for “additional preferences.”
Validation	Reject or coerce invalid input (unknown location, rating out of range); clear error messages for the UI/CLI.

Exit criteria: preferences deserialize from form/API/CLI args into one object used by the filter layer; validation errors are user-visible.

Implemented: package `src/milestone1/phase2_preferences/` (UserPreferences, preferences_from_mapping, optional allowed_city_names corpus check,

allowed_cities_from_restaurants). CLI: milestone1 prefs-parse ... (prints JSON or field errors on stderr).

Phase 3 — Integration layer (retrieval + prompt assembly)

Component	Responsibility
Deterministic filter	Apply hard filters first: location, min rating, budget, cuisine overlap—reduce to top N candidates (cap for LLM context, e.g. 15–50).
Ranking hint (optional)	Pre-sort by rating or composite score so the LLM sees a sensible default order even before reasoning.
Prompt builder	System + user messages (or single structured prompt) including: user preferences as JSON or bullets; candidate table as markdown/JSON; instructions to only recommend from the list; output format (see Phase 4).

Exit criteria: given preferences + loaded dataset, produce a stable (candidates[], prompt_payload) without calling the LLM yet; tests for filter edge cases (no matches, too many matches).

Implemented: package src/milestone1/phase3_integration/ (filter_and_rank, build_prompt_payload, build_integration_output). CLI: milestone1 prompt-build.

Phase 4 — Recommendation engine (LLM)

Concern	Approach
Model I/O	Thin client: temperature, max tokens, timeout; inject API key from environment.
Grounding	Prompt requires the model to cite restaurant names from the candidate list only; refuse or return empty if nothing fits.
Structured output	Ask for JSON (e.g. rankings[] with restaurant_id, rank, explanation) or strict markdown sections—then parse and validate.
Resilience	Retry on transient errors; fallback: return deterministic top- k with template explanations if the LLM fails.

Exit criteria: end-to-end call returns ranked items with explanations; parser validates structure; failures degrade gracefully.

Implemented: package `src/milestone1/phase4_llm/` (Groq OpenAI-compatible client, JSON rankings parse, deterministic fallback, `recommend_with_groq`). CLI: `milestone1 recommend`. Secrets: `GROQ_API_KEY` (see `.env.example`).

Phase 5 — Output and experience

Surface	Responsibility
Rendering	For each recommendation: name, cuisine, rating, estimated cost, AI explanation (per problem statement).
Empty states	“No restaurants match filters” vs “LLM could not justify picks”—different copy.
Observability (light)	Log latency, token usage if available, and filter counts (no PII in logs unless required).

Exit criteria: demo path from user input to readable results in one run; copy and layout match the minimum fields in the problem statement.

Implemented: package `src/milestone1/phase5_output/` (markdown/plain rendering, empty-state copy, stderr telemetry JSON). CLI: `milestone1 recommend-run` (end-to-end readable output + telemetry).

Phase 6 — Backend (HTTP API)

Concern	Approach
Role	Thin HTTP service that owns server-side secrets (<code>GROQ_API_KEY</code>), dataset access, and orchestration. The browser must not call Groq or Hugging Face directly.
Contract	Stable JSON request/response for “recommend”: preferences body aligned with Phase 2 keys; response carries ranked items (ids + display fields + explanations), source (llm / fallback / no_candidates), filter/candidate counts, and optional non-sensitive telemetry fields for the UI.
Endpoints (v1 intent)	POST <code>/api/v1/recommendations</code> (or equivalent) — validate input, run <code>load_restaurants</code> (with limits/caching policy), <code>recommend_with_groq</code> , return DTOs. GET <code>/health</code> — process up, keys configured (without exposing values). Optional: GET <code>/api/v1/meta</code> — e.g. sample <code>allowed_cities</code> cap for form hints.

Cross-cutting	Timeouts aligned with Phase 4; structured server logs (counts, latency, token totals—no raw user notes in info-level logs unless you explicitly choose to); CORS restricted to the dev frontend origin; request size limits on free-text fields (reuse Phase 2 max length).
Stack	Python-first is natural: e.g. FastAPI or Flask in src/ or a sibling package, sharing the installed milestone1 library. Alternative stacks (Node, etc.) are possible only if they duplicate contracts and call a Python sidecar—avoid unless required.

Exit criteria: frontend can complete one recommendation flow using only the API; API returns the same logical outcomes as milestone1 recommend / recommend-run for the same inputs (modulo caching).

Implemented: *pending* — document target layout here when added (e.g. src/milestone1/api/ or apps/api/).

Phase 7 — Frontend (web UI)

Concern	Approach
Role	Primary user-facing surface: preference form + results list, per phase0-scope.md.
Data flow	Browser only talks to the Phase 6 API. Map form fields to the API JSON schema (location, budget band, cuisines, minimum rating, optional additional text).
UI	Results show name, cuisines, rating, estimated cost, AI explanation for each row; reuse Phase 5 empty-state semantics (“no filter match” vs “model returned no grounded picks”) with clear, distinct copy.
UX	Loading states, validation errors inline, disabled submit while pending; optional “copy as Markdown” for demo.
Stack	Choose one and stay consistent: e.g. React + Vite (SPA) or HTMX + server templates (minimal JS). Host locally for milestone 1; no production SLA required in Phase 0.

Exit criteria: one demo path in the README: start API + UI, submit preferences, see ranked results or an intentional empty state.

Implemented: *pending* — e.g. apps/web/ or frontend/ + README section “Run the web app”.

Phase 8 — Deployment using Streamlit (optional)

Concern	Approach
Role	A single-process Python app (Streamlit) that exposes the same recommendation flow as the CLI/API: preferences in widgets → load corpus (Phase 1) → validate (Phase 2) → filter + prompt (Phase 3) → recommend_with_groq (Phase 4) → render ranked cards with explanations (Phase 5 semantics). No Node build and no separate SPA host required for this path.
Secrets	GROQ_API_KEY (and optional GROQ_MODEL) via Streamlit secrets (st.secrets) on Streamlit Community Cloud or via environment variables when self-hosting—same rules as Phase 6: keys never ship to the browser client bundle; Streamlit runs logic server-side .
Deployment (free tier)	Streamlit Community Cloud : connect the GitHub repo, set the main file path (e.g. streamlit_app.py or src/milestone1/phase8_streamlit/app.py), add secrets in the dashboard, deploy. Cold starts and resource limits apply on the free tier; keep load_limit / candidate_cap conservative. Alternatives: Docker image (streamlit run ...) on Render/Fly/other free allowances.
Relationship to Phase 6–7	Complementary : Phase 7 remains the primary product UI (browser + REST). Phase 8 is ideal for course demos , stakeholder previews , and fast sharing without operating Vite + CORS + two deployables. You may implement Streamlit without calling the HTTP API by importing milestone1 directly (duplication of orchestration is acceptable if thin); alternatively call POST /api/v1/recommendations if you want one orchestration path.
UX scope	Forms with st.selectbox / st.text_input / st.slider for location, cuisines, budget, minimum rating, and additional text; st.spinner while the model runs; st.expander for raw JSON or telemetry if useful. Match empty-state copy from Phase 5 where practical.

Exit criteria: README (or a short docs/streamlit-deploy.md) documents how to run locally (streamlit run ...) and how to deploy to Community Cloud (repo layout, secrets names, branch); a reviewer can open the hosted URL and complete one successful recommendation or see an intentional empty state.

Implemented: package src/milestone1/phase8_streamlit/ (app.py), repo root **streamlit_app.py** (Cloud endpoint), optional dependency **[streamlit]** in pyproject.toml, and streamlit-deploy.md.

Phase 8 — Deployment plan: Render (backend) + Vercel (frontend)

This document is the canonical guide for deploying Milestone 1 as **two independent services**:

- **Backend** — FastAPI app from src/milestone1/phase6_api/ on **Render**.
- **Frontend** — Vite + React SPA from frontend/ on **Vercel**.

The browser bundle is purely static and only talks to the Render URL over HTTPS. Provider keys (GROQ_API_KEY, optional HF_TOKEN) live **only** on Render.

0. One-time prep in the repo

The repo already builds cleanly for both targets — Render reads `pyproject.toml`, Vercel reads `frontend/package.json`. Two small additions make the deploy reproducible without clicking around dashboards.

0.1 Pin a Python version for Render

Add a `runtime.txt` at the repo root so Render uses Python 3.11 (matches `requires-python` in `pyproject.toml`):

```
python-3.11.9
```

0.2 Optional: `render.yaml` (Infrastructure-as-Code)

Render can read a `render.yaml` blueprint at the repo root to provision the service automatically:

services:

- type: web

- name: milestone1-api

- runtime: python

- plan: free

- buildCommand: pip install -e .

- startCommand: uvicorn milestone1.phase6_api.app:app --host 0.0.0.0 --port \$PORT

- healthCheckPath: /health

- autoDeploy: true

- envVars:

- key: GROQ_API_KEY

- sync: false

- key: GROQ_MODEL

- sync: false

- key: HF_TOKEN
sync: false
- key: CORS_ORIGINS
sync: false

sync: false keeps secret values out of the repo; you set them in the Render dashboard.

0.3 Optional: frontend/vercel.json

Vercel auto-detects Vite, but a small config makes SPA fallback explicit and avoids surprises if you add client-side routing later:

```
{  
  "buildCommand": "npm run build",  
  "outputDirectory": "dist",  
  "installCommand": "npm install",  
  "framework": "vite",  
  "rewrites": [  
    { "source": "/(.*)", "destination": "/index.html" }  
  ]  
}
```

1. Deploy the backend on Render

1.1 Create the service

1. Push the repo to GitHub.
2. Render dashboard → **New** → **Web Service** → connect the GitHub repo.
3. If you committed render.yaml, choose **Blueprint** and Render fills in the rest. Otherwise use the manual settings below.

1.2 Manual settings (if not using render.yaml)

Field	Value
Environment	Python 3
Region	nearest free region
Branch	main
Root Directory	<i>(blank — repo root)</i>
Build Command	pip install -e .
Start Command	uvicorn milestone1.phase6_api.app:app --host 0.0.0.0 --port \$PORT
Health Check Path	/health
Plan	Free (or paid for no cold starts)

milestone1.phase6_api.app:app exists at module scope in src/milestone1/phase6_api/app.py (app = create_app()), so uvicorn can import it without running the milestone1-api console script. The startup hook **prewarms** the city list in a background thread, so the first /api/v1/meta and /api/v1/recommendations calls do not pay the full Hugging Face load cost.

1.3 Environment variables on Render

Set in **Environment** → **Environment Variables**:

Var	Required?	Purpose
GROQ_API_KEY	yes	Phase 4 LLM calls. Get from https://console.groq.com/keys .
GROQ_MODEL	optional	Override default Groq model id (default: llama-3.3-70b-versatile).
HF_TOKEN	optional	Higher Hugging Face Hub rate limits when streaming the dataset.
CORS_ORIGINS	yes (after Vercel deploy)	Comma-separated list of allowed browser origins. See §3.
PORT	auto	Render injects this; the app reads it via uvicorn ... --port \$PORT.

Do **not** add API_HOST — the start command already binds 0.0.0.0.

1.4 Verify

After the first deploy, hit:

- `https://<service>.onrender.com/health` → `{"status":"ok","groq_configured":true}`
- `https://<service>.onrender.com/api/v1/meta?cities_cap=20` → JSON with a cities array
- `https://<service>.onrender.com/docs` → Swagger UI

Note the service URL — it goes into the Vercel build env next.

Cold starts: Render free-tier services sleep after ~15 minutes of inactivity. The first request after sleep can take 30–60 s. The Phase 6 prewarm thread reduces *post-startup* latency but does not eliminate the dyno boot itself. If demos need snappy first hits, upgrade the plan or hit `/health` from an uptime pinger (Better Stack / cron-job.org).

2. Deploy the frontend on Vercel

2.1 Create the project

1. Vercel dashboard → **Add New** → **Project** → import the same GitHub repo.
2. **Root Directory:** `frontend/` (critical — without this Vercel tries to build the Python project).
3. **Framework Preset:** Vite (auto-detected).
4. Build / Output should auto-fill from `package.json`:
 - **Install:** `npm install`
 - **Build:** `npm run build`
 - **Output:** `dist`

2.2 Environment variables on Vercel

Add under **Settings** → **Environment Variables**, scoped to **Production** (and **Preview** if you want previews to hit the same backend):

Var	Value
VITE_API_BASE_URL	<code>https://<your-render-service>.onrender.com</code> (no trailing slash)

Vite inlines `VITE_*` vars at build time, so a redeploy is needed to pick up changes (Vercel does this automatically on env-var save).

Never put `GROQ_API_KEY` in any `VITE_*` var — `frontend/src/lib/api.ts` only ever calls `${VITE_API_BASE_URL}/...`, and that boundary is what keeps provider keys server-side.

2.3 Verify

After deploy:

- `https://<project>.vercel.app/` loads the SPA.

- DevTools → Network → submit the form → request goes to `https://<render>.onrender.com/api/v1/recommendations` and returns 200.
 - If the request is blocked by the browser with a CORS error, you have not yet completed §3.
-

3. Wire CORS on Render to the Vercel origin

`src/milestone1/phase6_api/app.py` reads `CORS_ORIGINS` (comma-separated). Set it on Render to the exact origins the browser will use:

`CORS_ORIGINS=https://<project>.vercel.app,https://<project>-git-main-<team>.vercel.app`

Common gotchas:

- **No trailing slash, no path.** Origin only: `https://foo.vercel.app`, not `https://foo.vercel.app/`.
- **Custom domain?** Add it too:
`CORS_ORIGINS=https://app.example.com,https://<project>.vercel.app`.
- **Preview deploys** get unique subdomains. Either disable preview-env builds, point them at a separate staging Render service, or temporarily widen `CORS_ORIGINS` while testing — never to `*` for a credentialed app.

After saving the env var, Render restarts the service. Re-test the SPA call from the browser.

4. Smoke-test checklist

Run these in order from the deployed Vercel URL:

1. Page loads, hero + form render, no console errors.
2. GET `/api/v1/meta` populates the city dropdown (visible on first paint, served by Render).
3. Submit form with a valid city → status badge shows source: llm and ranked cards render.
4. Submit with an obviously empty filter combo (e.g. min rating 5 + a quiet city) → renders the **no candidates** empty state copy from Phase 5.
5. Tail Render logs (Logs tab) — request lines appear with 200, telemetry JSON is logged on stderr.

If any step fails, see §5.

5. Troubleshooting

Symptom	Likely cause / fix
---------	--------------------

Browser shows CORS error	CORS_ORIGINS on Render does not include the exact Vercel origin. Update env var, wait for restart.
Failed to fetch from frontend	VITE_API_BASE_URL missing or wrong. Confirm value, then redeploy on Vercel.
groq_configured: false from /health	GROQ_API_KEY not set on Render, or has whitespace. Re-paste, redeploy.
First request hangs ~30 s	Render free-tier cold start. Ping /health first, or upgrade plan.
/api/v1/meta 500s with HF errors	Hugging Face throttle. Set HF_TOKEN on Render, or lower load_limit.
Vercel build fails on tsc --noEmit	Same TS error you would see locally — fix in frontend/, push, Vercel rebuilds.
Render build fails on pip install -e .	Confirm runtime.txt is python-3.11.x; Render's default Python may be too old.

6. Rollback

- **Backend:** Render keeps a deploy history; **Manual Deploy** → **Rollback** to a previous build.
- **Frontend:** Vercel's **Deployments** tab → **Promote to Production** on a known-good build.

Both platforms support instant rollback without rebuilding.

7. Cost shape (free-tier)

Resource	Free tier	Notes
Render web service	750 hrs/month	Sleeps when idle (cold starts).
Vercel hobby	100 GB bandwidth, 6k build min/month	Static SPA is essentially free at this scale.
Groq	Free dev quota	Keep candidate_cap modest in the API request body.

Hugging Face
Hub

Anonymous

Add HF_TOKEN if you hit rate limits.

For demos and coursework, the free tiers are sufficient. For a graded review, hit /health once before the demo to wake the Render dyno.

Phase 9 — Hardening and handoff (optional but recommended)

- Automated tests for filters, prompt shape, JSON parsing (fixtures with fake LLM responses), and **API contract** tests (golden JSON for happy/empty/error paths).
- README: install, set **GROQ_API_KEY**, run API + UI, CLI fallbacks, and limitations (dataset revision, rate limits, candidate cap).
- Cost/latency notes: candidate cap, model id, when to raise load limits, caching strategy for repeated queries (optional in-process LRU of recent Hub windows—only if measured need).

STEPS

STEP 1 → Created a doc folder inside which we added a [problemstatement.md](#)

STEP 2 → Generating the architecture

STEP 3 → Generating the edge cases

Prompts

Prompt 1 → Write @docs/problemstatement.md in better way

Prompt 2 → Create a phase wise architecture for this @docs/problemstatement.md

Prompt 3 → Generate the detailed edge cases for this project using @docs/problemstatement.md and @docs/phased-architecture.md

Prompt 4 → Implement phase0 as per the @docs/phased-architecture.md

Prompt 5 → In phase0 we can update that basic web UI will be the source of input

Prompt 6 → Implement phase1 in the separate folder as per the @docs/phased-architecture.md

Prompt 7 → run phase1 so that data is downloaded

Prompt 8 → Implement phase2 in the separate folder as per the @docs/phased-architecture.md

Prompt 9 → Implement phase3 as per the @docs/phased-architecture.md in the separate folder

Prompt 10 → Implement phase4 as per the @docs/phased-architecture.md in the separate folder, LLM used in this phase will be Groq.

STEPS for creating API KEY on GROQ →

- Create account on Groq <https://groq.com/>
- Created an API KEY on <https://console.groq.com/keys>
- Add API key in ENV file

Create an .env file YOURSELF

GROK_API_KEY = <api-key>

Prompt 11 → Test phase4 will a live example like this below - Input is Bellandur, Budget is 2000, rating is 4.0

Get the top5 restaurants from the LLM

Prompt 12 → Run the above command and tell me exact output from LLM

Prompt 13 → Update the architecture after phase5 so that proper backend and frontend we have for this project

Prompt 14 → Implement phase6 as per the @docs/phased-architecture.md

FRONTEND frameworks

- [Next.js](#)
- [React.js](#)
- Angular / Vite

Designing Tool →

- <https://stitch.withgoogle.com/>
- <https://godly.website/>

Prompt 17 → Implement phase7 as per the @docs/phased-architecture.md in Next.js

Prompt 18 → Run both backend and frontend

Test it locally

Prompt 19 → In frontend rating filter is not present we provide user to add minimum rating as well

Prompt 20 → Stop backend and frontend

Prompt 21 → How can we deploy this project using free tools ?

Prompt 22 → In @docs/phased-architecture.md add one more phase for deployment using streamlit

Prompt 23 → Implement phase8 as per the @docs/phased-architecture.md

Prompt 24 → Push the changes to <URL>

Prompt 25 → load_limit in the frontend in streamlit we don't want we can remove it we can remove the max cities as well

Advance model tuning is not required; we can remove it.

Frontend on streamlit can be improved having better fonts and components

OTHER AI TOOLS

(If limits are exhausted)

- Cursor
- [Google Antigravity](#)
- [Windsurf](#)
- [Qoder](#)

Deployment

- Develop it locally (in your PC)
- Deployment CHANGES for XYZ platform
- Push the code to Github
- Actual Deployment (Currently we have done deployment on streamlit)
- Tools
- <https://share.streamlit.io/> (ONLY basic frontend will come up)

For good frontend we will use →

- <https://dashboard.render.com/> (Backend)
- <https://railway.com/> (Backend)
- <https://vercel.com/> (Frontend)

Two deployment Prompt

- We don't want to deploy on streamlit, we want two deployments: backend on render and frontend on vercel. First we can remove streamlit changes and then we can generate a deployment plan for render and VERCEL.
- Do backend changes of **render/ railway** as per the @docs/[deployment.md](#)
- Then deploy backend
- Do frontend changes for **vercel** as per the @docs/[deployment.md](#)
- Then deploy frontend

STEPS for render

- Push these files to GitHub.
- Render → New → Blueprint → pick the repo. It will read `render.yaml` and provision `milestone1-api` automatically.
-

How on be track

- <https://www.youtube.com/@AndrejKarpathy>
- <https://www.youtube.com/@nicksaraev>
- <https://www.youtube.com/@nateherk>
- Anthropic (Linkedin)